Chapter 4

Monte Carlo Optimization

Contents

4.1 Optimization problems in Bayesian inference 50
4.2 Stochastic search methods 51
4.3 Simulated annealing 53
4.4 The EM algorithm 55
4.5 Stochastic approximation 57

Chapters 1 to 3 were about computing integrals: posterior means, probabilities, and evidences. Nevertheless, many practical questions ask for the **best value** instead, such as the most probable conversion rate, the budget split that maximizes a campaign's expected benefit, and the bid that wins a given share of auctions.
In this chapter, we focus on optimization problems that are hard for one of three reasons, and we will answer each of them with simulations.

1. The objective is an *expectation* that we can only estimate by Monte Carlo (Section 4.2).
2. The objective has several *local optima*, and a local method gets trapped (Section 4.3).
3. The objective involves *latent variables* or can only be *observed with noise* (Sections 4.4 and 4.5).

Throughout this chapter, we keep the campaign of Chapter 1 as our running example.

4.1 Optimization problems in Bayesian inference

Example 4.1 (The most probable conversion rate)**.** The conjugate posterior of the campaign, $\mathrm{Beta}(14, 226)$, is highest at its mode $\dfrac{(a-1)}{(a+b-2)} = \dfrac{13}{238} \approx 0.0546$, slightly below its mean $\dfrac{14}{240} \approx 0.0583$ and the MLE 0.06.
For the non-conjugate log-odds model described in Example 1.4, there is no formula: a one-dimensional numerical optimizer finds the mode of the posterior density of η at $\hat{\eta} = -2.8029$, that is, $\theta = \sigma(\hat{\eta}) = 0.0572$.

Surprisingly, if we instead maximize the posterior density *of* θ, we obtain 0.0539, a different value. The density of $\theta = \sigma(\eta)$ is the density of η multiplied by the Jacobian $|\mathrm{d}\eta/\mathrm{d}\theta| = 1/\{\theta(1-\theta)\}$ (Proposition 2.1), which shifts the mode. Hence, the posterior mean, 0.0576, does not depend on the scale on which it is computed.

Definition 4.1 (Maximum a posteriori estimate)**.** The **maximum a posteriori (MAP)** estimate is a mode of the posterior density,

$$\begin{aligned}\hat{\theta}_{\text{MAP}} &= \arg\max_{\theta\in\Theta}\ \pi(\theta\,|\,\mathbf{y}) \\ &= \arg\max_{\theta\in\Theta}\ \{\log p(\mathbf{y}\,|\,\theta) + \log\pi(\theta)\}.\end{aligned}$$

The evidence $p(\mathbf{y})$ does not depend on θ, so the MAP only requires the unnormalized posterior. With a constant (i.e., flat) prior, it coincides with the MLE. Optimization problems appear throughout Bayesian practice:

Point estimates: the MLE and the MAP estimate;

The starting point and the scale of a sampler: the Laplace proposal of Example 3.4 was centered at the posterior mode;

Decisions: choose the action x that maximizes the posterior expected utility $\mathbb{E}[u(x,\theta)\,|\,\mathbf{y}]$, an integral that must itself be estimated (Section 4.2);

hyperparameters: the choice of hyperparameters by maximizing the evidence $p(\mathbf{y})$ (empirical Bayes).

When the objective is smooth and has a single maximum, deterministic optimizers (e.g., Newton's method, BFGS, Brent's method in one dimension) are fast and accurate. We are interested in the cases where they are not enough.

Example 4.2 (Purchase amounts with unknown shape)**.** In Example 1.3 the shape of the Gamma model of purchase amounts was known. With both the shape α and the scale β unknown, the score equation for β still gives $\beta = \bar{y}/\alpha$; substituting it, the score equation for α becomes

$$\log\alpha - \psi(\alpha) = \log\bar{y} - \overline{\log y},$$

where $\psi = (\log\Gamma)'$ is the digamma function and $\overline{\log y}$ the average of the logarithms of the amounts. It has no closed-form solution. For the ten amounts, the right-hand side is 0.1099, and a root-finding algorithm gives $\hat{\alpha} = 4.708$ and $\hat{\beta} = 12.74$ euros (i.e., mean $\hat{\alpha}\hat{\beta} = 60$, as before). The log-likelihood rises from -48.18 at $(\alpha,\beta) = (2, 30)$ to -46.64 at the MLE.

4.2 Stochastic search methods

Example 4.3 (Splitting a budget across channels)**.** The retailer splits a weekly budget of $B = 100$ (thousand euros) across TV, search, and social media. With spend x_c on channel c, the sales lift is

$$L(x) = \sum_c \beta_c \frac{x_c}{\kappa_c + x_c},$$

with the saturation of Section 1.3.1, half-saturation spends $\kappa = (40, 10, 20)$, and uncertain effects: the posterior from past campaigns is approximately $\beta_c \sim \mathcal{N}(m_c, s_c^2)$ with $m = (30, 12, 18)$ and $s = (12, 2, 8)$. TV has the largest expected effect but also the largest uncertainty. A risk-averse manager maximizes the posterior **expected utility**

$$U(x) = \mathbb{E}[1 - e^{-aL(x)}\,|\,\mathbf{y}], \qquad a = 0.05, \tag{4.1}$$

over the allocations with $x_1 + x_2 + x_3 = B$. The utility $1 - e^{-aL}$ grows with the lift but penalizes uncertain outcomes.

The objective is an expectation, which we estimate with $M = 2000$ posterior draws of β.

Random search draws candidate allocations uniformly on the set of possible splits, estimates U at each, and keeps the best. Because $L(x)$ is Normal given x, the exact value of U is also available here, $U(x) = 1 - \exp\{-a\mu(x) + a^2\sigma^2(x)/2\}$ with μ and σ^2 the mean and variance of $L(x)$, which lets us check the search.

The equal split gives $U = 0.8049$; the optimum, $(49.5, 20.5, 30.0)$, gives $U = 0.8149$.

Candidates N	**Best found allocation**	**Exact** U	**Gap to optimum**
10	$(50.7, 30.4, 18.8)$	0.8077	0.0071
100	$(54.5, 15.9, 29.6)$	0.8137	0.0012
1 000	$(49.4, 21.3, 29.2)$	0.8148	0.0001
10 000	$(49.1, 20.5, 30.4)$	0.8149	0.0000

The optimal split gives TV half of the budget despite its uncertainty, and keeps a fifth for search, whose effect is small but well known.

In general, we want to maximize an objective that is an expectation,

$$h(x) = \mathbb{E}\big[H(x, \xi)\big], \qquad \hat{h}_M(x) = \frac{1}{M}\sum_{m=1}^{M} H(x, \xi^{(m)}), \tag{4.2}$$

where ξ is random (i.e., here, the channel effects) and $\hat{h}_M$ is its Monte Carlo estimate. Two practical rules follow from Chapter 3:

- **Common random numbers.** Compare candidates with the **same** draws $\xi^{(1)}, \dots, \xi^{(M)}$. The Monte Carlo errors are then strongly positively correlated, and the **differences** $\hat{h}_M(x) - \hat{h}_M(x')$, which decide which candidate wins, are much more accurate than either estimate. In Example 4.3 every estimate is slightly too low (the best Monte Carlo value is 0.8136, its exact value 0.8149), yet the ranking of the candidates is right.
- **Winner's curse.** With independent draws for each candidate, the best of many noisy estimates tends to overestimate its true value, and the winner is often just a lucky draw. Common random numbers and a re-evaluation of the winner with fresh draws protect against it.

Proposition 4.1 (Random search)**.** *Let $x^{(1)}, \dots, x^{(N)}$ be drawn independently from a distribution that gives probability $p_\varepsilon > 0$ to the set A_ε of points within ε of the maximum value of h. Then the best candidate lies in A_ε with probability $1 - (1 - p_\varepsilon)^N \to 1$.*

Proof. No candidate is in A_ε with probability $(1 - p_\varepsilon)^N$, by independence. □

Random search always works in principle, but p_ε shrinks exponentially with the dimension of x: the curse of dimensionality again! Using the shape of h is much more efficient. Under mild conditions, derivative and expectation can be exchanged,

$$\nabla h(x) = \mathbb{E}\big[\nabla_x H(x, \xi)\big],$$

so an unbiased estimate of the gradient is an average of $\nabla_x H(x, \xi^{(m)})$ over a few draws. **Stochastic gradient ascent** moves uphill along such noisy gradients,

$$x_{k+1} = x_k + \gamma_k \, \widehat{\nabla h}(x_k), \tag{4.3}$$

with step sizes γ_k that decrease slowly enough to keep moving and fast enough to average out the noise (Section 4.5).

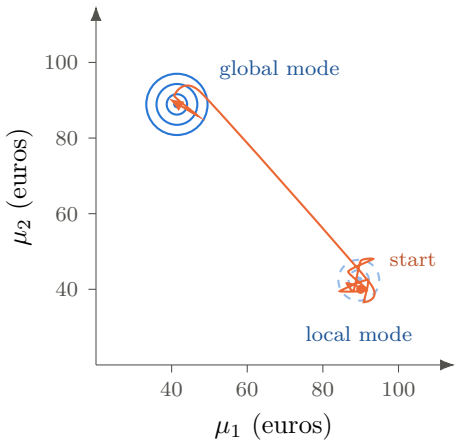


Figure 4.1: Posterior of the two segment means (Example 4.5): the global mode (solid contours) and the local mode (dashed). Simulated annealing started at $(90, 40)$ (path every 200 iterations) wanders near the local mode and then jumps to the global one.

Example 4.4 (Stochastic gradient ascent for the budget)**.** Let write the allocation as $x = B\,\mathrm{softmax}(z)$, where $\mathrm{softmax}(z)_c = \dfrac{e^{z_c}}{\sum_{c'} e^{z_{c'}}}$, so that every $z \in \mathbb{R}^3$ gives a valid split.

The gradient of (4.1) with respect to x_c is $\mathbb{E}[a\, e^{-aL} \beta_c\, \kappa_c/(\kappa_c + x_c)^2]$, estimated at each step with a fresh minibatch of 50 posterior draws, and the chain rule through the softmax gives the gradient in z.

With $\gamma_k = 10/k^{0.6}$, starting from the equal split, the allocation is $(48.5, 21.4, 30.1)$ after 10 steps ($U = 0.8148$) and $(50.1, 20.3, 29.6)$ after 100 steps ($U = 0.8149$, the optimum to four digits), using 5 000 draws in total against 2 million for the random search with 1 000 candidates.

4.3 Simulated annealing

Example 4.5 (Two segments of unequal size)**.** Returning to the two customer segments of Example 1.5, now with unequal shares: 30% bargain hunters and 70% premium buyers ($\sigma = 10$ euros, prior $\mathcal{N}(65, 30^2)$ on each mean, 100 simulated purchase amounts). The labels are no longer interchangeable, and the posterior of (μ_1, μ_2) has a *global* mode at $(41.4, 88.9)$ and a *local* mode near the swapped point $(89.5, 42.4)$, with log posterior densities -443.99 and -466.40: the global mode is $e^{22.4} \approx 5 \times 10^9$ times more probable. Gradient ascent climbs to whichever mode is closer: from 20 random starting points it finds the global mode 40% of the time. Figure 4.1 shows simulated annealing escaping from the local mode.

Simulated annealing (Kirkpatrick et al., 1983) borrows its idea from metallurgy: a metal cooled slowly settles in its lowest-energy configuration. It proposes random moves and accepts a worse point with a probability that depends on a **temperature** T_k, decreasing over time. An uphill move is always accepted; a downhill move of size $\Delta < 0$ is accepted with probability e^{Δ/T_k}: often when T_k is large, rarely when T_k is small. At a fixed temperature, Algorithm 3 is the Metropolis algorithm (Chapter 6) targeting the **Boltzmann distribution**

$$\pi_T(x) \propto \exp\{h(x)/T\}. \tag{4.4}$$

Proposition 4.2 (Concentration of the Boltzmann distribution)**.** *Let h have a unique global maximizer $x^\star$ on a finite set (or, on $\mathbb{R}^d$, a continuous h with integrable $e^{h/T}$). As $T \to 0$, π_T concentrates on $x^\star$: for every point x with $h(x) < h(x^\star)$, $\pi_T(x)/\pi_T(x^\star) = e^{-(h(x^\star)-h(x))/T} \to 0$.*

Algorithm 3: Simulated annealing for maximizing h

Input: objective h, starting point x_0, symmetric proposal $q(\cdot \mid x)$, temperatures $T_1 > T_2 > \dots$

for $k = 1, 2, \dots, K$ **do**
  draw a proposal $y \sim q(\cdot \mid x_{k-1})$ and $U \sim \mathcal{U}(0,1)$;
  if $U \le \exp\{(h(y) - h(x_{k-1}))/T_k\}$ **then**
    $x_k \leftarrow y$;
  else
    $x_k \leftarrow x_{k-1}$;
return the best point visited;

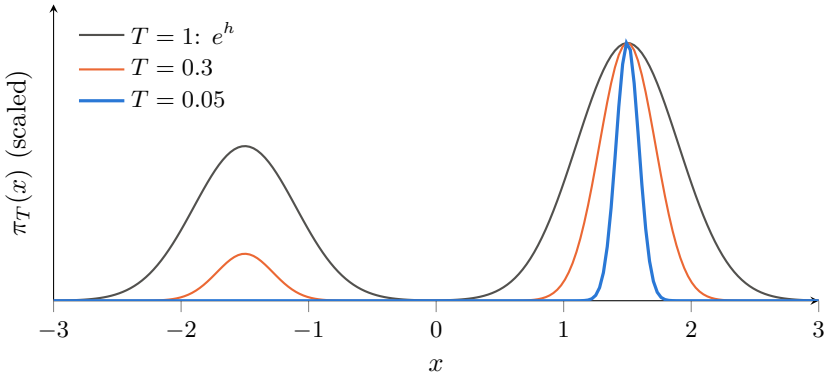


Figure 4.2: Boltzmann distributions $\pi_T \propto e^{h/T}$ of a bimodal objective, scaled to a maximum of one. As the temperature decreases, the mass leaves the lower mode and concentrates on the global maximizer.

For the two segments, the ratio of the Boltzmann weights at the two modes is $e^{22.4/T}$: 9.4 at $T = 10$, about 1.7×10^3 at $T = 3$, and 5×10^9 at $T = 1$ (Figure 4.2). At high temperature the chain moves freely between the modes; as the temperature drops, it settles in the highest one. The cooling must be slow enough for the chain to have visited the global mode before the barriers become impassable. A logarithmic schedule $T_k = c/\log(1+k)$ with c larger than the depth of the deepest non-global valley guarantees convergence to the global maximum (Robert and Casella, 2004); in practice, faster geometric schedules $T_k = T_0 \rho^k$ are common. For the segments, with $T_k = 30/\log(1+k)$ and Gaussian proposals of standard deviation 20, annealing reaches the global mode from 90% of the 20 starting points, against 40% for gradient ascent.

Example 4.6 (Scheduling a campaign over 12 weeks)**.** Let us choose which of 12 weeks to advertise, $x_t \in \{0, 1\}$, with a spend of 10 in an active week. The sales respond through carryover (i.e., adstock decay $\lambda = 0.5$) and saturation ($\kappa = 15$, $\beta = 40$), and audiences tire of the same advertisement: the r-th consecutive active week is only 0.6^r as effective (**wear-out**). Each active week costs 10. The net contribution is

$$h(x) = \sum_{t=1}^{12} \beta \frac{a_t}{\kappa + a_t} - 10 \sum_{t=1}^{12} x_t, \qquad a_t = 10\, x_t\, 0.6^{r_t} + \lambda a_{t-1},$$

where r_t is the number of consecutive active weeks just before week t. With $2^{12} = 4096$ schedules we can enumerate them all: the best is the **pulsing** schedule 101010101010 (one week on, one week off), worth 120.09, while advertising every week loses money (−30.69). Enumeration also reveals 31 local maxima: schedules that no single change of week can improve. A greedy method that flips one week at a time while it improves the objective never finds the

optimum from 50 random starting schedules; simulated annealing with single-week flips and geometric cooling $T_k = 10 \times 0.999^k$ finds it in 96% of the runs.

4.4 The EM algorithm

Example 4.7 (Estimating the two segments)**.** Let us keep the 100 purchase amounts of Example 4.5, but now estimate by maximum likelihood the share w of the first segment and both means ($\sigma = 10$ known). The log-likelihood

$$\ell(w, \mu_1, \mu_2) = \sum_{i=1}^{n} \log\{ w\,\mathcal{N}(y_i; \mu_1, \sigma^2) + (1-w)\,\mathcal{N}(y_i; \mu_2, \sigma^2) \}$$

has no closed-form maximizer. But if we knew which segment each buyer belongs to, the MLE would be simple averages: the share of segment 1 and the mean amount within each segment. The **expectation–maximization (EM)** algorithm alternates between guessing the segments and re-estimating:

- (E) compute each buyer's probability of belonging to segment 1 under the current parameters, the **responsibility**

$$r_i = \frac{w\,\mathcal{N}(y_i; \mu_1, \sigma^2)}{w\,\mathcal{N}(y_i; \mu_1, \sigma^2) + (1-w)\,\mathcal{N}(y_i; \mu_2, \sigma^2)};$$

- (M) re-estimate with weighted averages: $w = \frac{1}{n}\sum_i r_i$, $\mu_1 = \sum_i r_i y_i / \sum_i r_i$, $\mu_2 = \sum_i (1-r_i) y_i / \sum_i (1-r_i)$.

Starting from $(w, \mu_1, \mu_2) = (0.5, 50, 60)$:

Iteration	w	μ_1	μ_2	Log-likelihood
0	0.500	50.00	60.00	−707.68
1	0.305	43.40	84.29	−441.91
2	0.352	40.91	88.57	−433.99
3	0.360	41.37	88.94	−433.87
5	0.363	41.51	89.05	−433.86
10	0.363	41.52	89.05	−433.86

The log-likelihood increases at every iteration (Figure 4.3) and the estimates settle after five iterations, close to the values used to simulate the data ($w = 0.3$, 40, 90; the realized share of segment 1 in the sample is 0.37).

The general setting has observed data $\mathbf{y}$, **latent** (i.e., unobserved) variables z, in our case, the segment of each buyer, and a model $p(\mathbf{y}, z \mid \vartheta)$ whose **complete-data log-likelihood** $\log p(\mathbf{y}, z \mid \vartheta)$ is easy to maximize. The observed-data likelihood integrates z out, $p(\mathbf{y} \mid \vartheta) = \int p(\mathbf{y}, z \mid \vartheta)\, \mathrm{d}z$, as in (1.12).

Definition 4.2 (EM algorithm)**.** Given the current value ϑ_k, the EM algorithm (Dempster et al., 1977) computes

$$\text{(E-step)} \quad Q(\vartheta \mid \vartheta_k) = \mathbb{E}\big[\log p(\mathbf{y}, z \mid \vartheta) \mid \mathbf{y}, \vartheta_k\big] = \int \log p(\mathbf{y}, z \mid \vartheta)\, p(z \mid \mathbf{y}, \vartheta_k)\, \mathrm{d}z,$$

$$\text{(M-step)} \quad \vartheta_{k+1} = \arg\max_{\vartheta} \; Q(\vartheta \mid \vartheta_k).$$

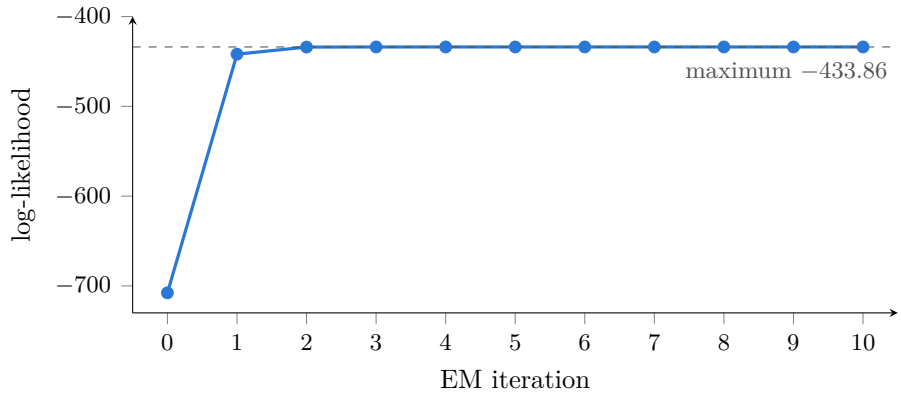


Figure 4.3: Log-likelihood along the EM iterations of Example 4.7: it never decreases and reaches its maximum after a few iterations.

For the segments, the complete-data log-likelihood is linear in the indicators $\mathbf{1}\{z_i = 1\}$, so the E-step replaces each indicator by its conditional expectation r_i, and the M-step gives the weighted averages above.

Proposition 4.3 (Monotonicity of EM)**.** *Each EM iteration does not decrease the observed-data log-likelihood:* $\log p(\mathbf{y} \mid \vartheta_{k+1}) \geq \log p(\mathbf{y} \mid \vartheta_k)$.

Proof. Write $\log p(\mathbf{y} \mid \vartheta) = \log p(\mathbf{y}, z \mid \vartheta) - \log p(z \mid \mathbf{y}, \vartheta)$ and take the expectation over $z \sim p(z \mid \mathbf{y}, \vartheta_k)$: $\log p(\mathbf{y} \mid \vartheta) = Q(\vartheta \mid \vartheta_k) - H(\vartheta \mid \vartheta_k)$ with $H(\vartheta \mid \vartheta_k) = \mathbb{E}[\log p(z \mid \mathbf{y}, \vartheta) \mid \mathbf{y}, \vartheta_k]$. By Jensen's inequality, $H(\vartheta \mid \vartheta_k) \leq H(\vartheta_k \mid \vartheta_k)$ for every ϑ (the Kullback–Leibler divergence between the two conditional distributions of z is non-negative). The M-step gives $Q(\vartheta_{k+1} \mid \vartheta_k) \geq Q(\vartheta_k \mid \vartheta_k)$. Subtracting, $\log p(\mathbf{y} \mid \vartheta_{k+1}) - \log p(\mathbf{y} \mid \vartheta_k) \geq 0$. □

EM converges to a *stationary point* of the likelihood, which depends on the starting value: a local maximum, or occasionally a saddle point. Several starting points remain good practice. When the E-step expectation has no closed form, it can be replaced by a Monte Carlo average.

Definition 4.3 (Monte Carlo EM)**.** In **Monte Carlo expectation–maximization (MCEM)** (Wei and Tanner, 1990), the E-step draws $z^{(1)}, \ldots, z^{(m)}$ from $p(z \mid \mathbf{y}, \vartheta_k)$ and uses $\hat{Q}(\vartheta \mid \vartheta_k) = \dfrac{1}{m} \displaystyle\sum_{j=1}^{m} \log p(\mathbf{y}, z^{(j)} \mid \vartheta)$.

Example 4.8 (Censored session times)**.** Session durations on the website follow an exponential distribution with mean $1/\lambda$, but the tracking tool records at most $c = 10$ minutes: longer sessions appear as "10+" (**censored** observations). Of 200 simulated sessions (true mean of 4 minutes), 16 are censored. Treating "10+" as exactly 10 underestimates the mean (3.73 minutes). The latent variables are the true durations of the censored sessions. By the memorylessness of the exponential distribution, a censored duration is c plus an exponential with the same rate, so the E-step replaces it by $c + 1/\lambda_k$ and the M-step sets $1/\lambda_{k+1}$ to the average completed duration. Starting from a mean of 1 minute, EM method gives us 3.812, 4.037, and then 4.056 minutes after five iterations: the MLE.
MCEM replaces $c + 1/\lambda_k$ by the average of m simulated durations $c + \mathrm{Exp}(\lambda_k)$. Over 200 runs of 40 iterations, the final estimates have mean 4.046 and standard deviation 0.086 with $m = 1$, 0.025 with $m = 10$, and 0.009 with $m = 100$: the Monte Carlo error decreases like $1/\sqrt{m}$, and the iterates fluctuate around the MLE instead of converging to it exactly.

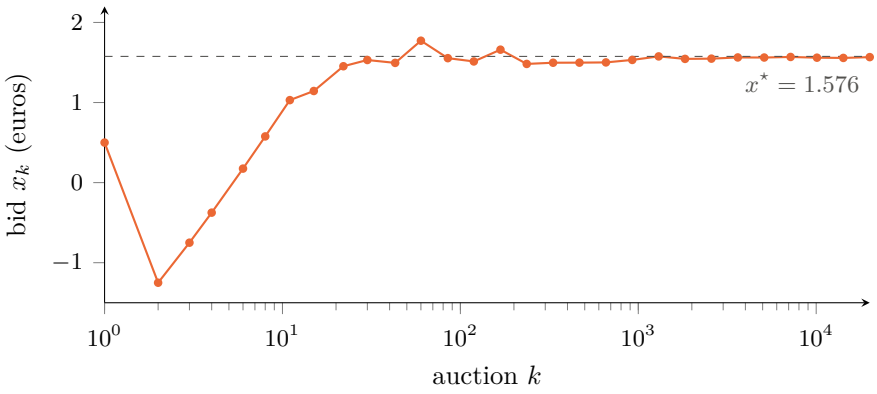


Figure 4.4: Robbins–Monro recursion (4.5) for the bid that wins 30% of the auctions, on a logarithmic horizontal axis: large early steps find the region quickly, decreasing steps average out the noise.

4.5 Stochastic approximation

Example 4.9 (The bid that wins 30% of auctions)**.** The retailer buys advertising slots in auctions. A bid x, in euros, wins with probability $p(x)$, which is unknown, and each auction only reveals a win ($Y = 1$) or a loss ($Y = 0$). The goal is the bid $x^\star$ that wins 30% of the auctions, a root of $p(x) - 0.30$ that we can only observe with noise. The rule "raise the bid after a loss, lower it after a win",

$$x_{k+1} = x_k + \gamma_k \, (0.30 - Y_k), \qquad \gamma_k = \frac{5}{k}, \tag{4.5}$$

moves the bid up by $0.3\gamma_k$ after a loss and down by $0.7\gamma_k$ after a win; on average it moves up when $p(x_k) < 0.3$ and down otherwise. Figure 4.4 illustrates that, in the simulation, the true win curve is $p(x) = \sigma((x-2)/0.5)$, so that $x^\star = 1.576$. From $x_1 = 4$, the bid is 0.894 after 10 auctions, 1.521 after 100, 1.513 after 1 000 and 1.567 after 20 000.

Stochastic approximation finds a root of a function $g(x) = \mathbb{E}[G(x, \xi)]$ that we can only evaluate with noise. Given a noisy evaluation $G_k = G(x_k, \xi_k)$, the **Robbins–Monro** recursion (Robbins and Monro, 1951) is

$$x_{k+1} = x_k + \gamma_k \, G_k. \tag{4.6}$$

Theorem 4.1 (Robbins–Monro)**.** *Let g have a unique root $x^\star$ with $g(x) > 0$ for $x < x^\star$ and $g(x) < 0$ for $x > x^\star$, let the noise $G_k - g(x_k)$ have mean zero and bounded variance, and let $|g(x)|$ grow at most linearly. If the step sizes satisfy*

$$\sum_{k=1}^{\infty} \gamma_k = \infty \qquad \textit{and} \qquad \sum_{k=1}^{\infty} \gamma_k^2 < \infty, \tag{4.7}$$

then $x_k \to x^\star$ with probability one.

The first condition lets the iterates travel any distance. The second makes the accumulated noise $\sum \gamma_k (G_k - g(x_k))$ converge.
Steps $\gamma_k = \gamma_0 / k^\alpha$ satisfy both for $1/2 < \alpha \le 1$. The constant γ_0 matters in practice: over 200 repetitions of 2 000 auctions, $\gamma_k = 5/k$ gives a RMSE of 0.031, whereas $\gamma_k = 0.5/k$ gives 0.438, because the steps shrink before the bid has reached $x^\star$. Slower steps $\gamma_k = 5/k^{0.6}$ combined with the average of the iterates (i.e., **Polyak–Ruppert averaging** (Polyak and Juditsky, 1992)) are robust to this choice and give an RMSE of 0.035. Stochastic gradient ascent (4.3) is the Robbins–Monro recursion applied to $g = \nabla h$.

Example 4.10 (Stochastic gradient for a logistic regression)**.** The MLE of a logistic regression solves the score equation $\sum_{i=1}^{n} x_i\{y_i - \sigma(x_i^\top \beta)\} = 0$, a root-finding problem. On the two-channel data of the Lab 3 (2 000 visitors, exposure to TV and search), a deterministic optimizer gives $\hat{\beta} = (-3.276, 0.143, 1.023)$ for the intercept, TV and search. **Stochastic gradient descent (SGD)**, in our case, ascent on the log-likelihood, uses at each step the score of a random minibatch of 32 visitors, an unbiased estimate of the full score up to a constant, with $\gamma_k = 2/k^{0.6}$. After 20 000 steps the average of the last 10 000 iterates is $(-3.187, 0.106, 0.931)$: close to the MLE, at a fraction of the cost per step, but with a visible remaining error. This trade-off is why SGD dominates for very large data sets.

Summary

Takeaways

- Bayesian practice needs optimization: MAP and MLE estimates, starting points and scales of samplers, and decisions that maximize a posterior expected utility. The MAP depends on the parametrization; the posterior mean does not.
- When the objective is an expectation, estimate it by Monte Carlo with common random numbers; random search always works but scales badly, stochastic gradients use the shape of the objective.
- Simulated annealing accepts downhill moves with probability $e^{\Delta/T}$ and lowers the temperature: the Boltzmann distribution $\pi_T \propto e^{h/T}$ concentrates on the global maximum as $T \to 0$.
- EM maximizes likelihoods with latent variables by alternating conditional expectations and complete-data maximizations; it never decreases the likelihood, but it finds a stationary point that depends on the start. MCEM simulates the E-step.
- Robbins–Monro finds roots of noisy functions with steps satisfying $\sum \gamma_k = \infty$ and $\sum \gamma_k^2 < \infty$; stochastic gradient methods are its special case.

Further reading

Chapter 5 of Robert and Casella (2004) covers Monte Carlo optimization, including stochastic search, simulated annealing, and EM with its Monte Carlo variants. The EM algorithm is presented with mixture models in Chapter 9 of Bishop (2006) and Chapter 11 of Murphy (2012); the original references are Dempster et al. (1977) and, for MCEM, Wei and Tanner (1990). Simulated annealing was introduced by Kirkpatrick et al. (1983), and stochastic approximation by Robbins and Monro (1951), with averaging by Polyak and Juditsky (1992).